

FraudShield AI — Implementation & Fix Report

Complete technical log of environment setup, ML ensemble training, full-stack debugging, role-based access repairs, and GitHub deployment.

PROJECT NAME	REPOSITORY	BACKEND TEST SUITE
FraudShield AI / Credit Card Fraud Detection	github.com/KISHORE-NANDHAN/Credit-Card-Fraud-Detection	94 / 94 Passed (100%)
FRONTEND TEST SUITE		
16 / 16 Passed (100%)		

1. Executive Summary

The objective of this engagement was to pull the upstream FraudShield AI project, configure both backend and frontend development environments, resolve critical runtime inference failures (503 Model Not Ready / missing ML artifacts), eliminate frontend 404/403 authorization and routing bugs, verify end-to-end functionality via automated test suites, and deploy the fully operational solution to the user's personal GitHub repository.

Mission Accomplished: All 94 backend pytest tests and 16 frontend Vitest tests pass with 0 errors. The multi-model Stacking Ensemble (Extra Trees + Deep MLP + XGBoost Meta-Learner) was trained on the complete 284,807-transaction dataset, serialized into production artifacts, and validated in real time. The repository has been reinitialized and pushed cleanly to GitHub.

2. Root Cause Analysis & Machine Learning Resolution

The primary operational failure in the initial codebase was that `backend/app/ml/inference/predict.py` threw a `503 ModelNotReadyError` on every prediction or transaction creation request.

The Root Problem

- Missing Production Artifacts:** The inference service required 4 core artifacts (`scaler.pkl` , `extra_trees.pkl` , `mlp.keras` , `xgboost_meta.pkl`) and `schema.json` inside `backend/app/ml/models/saved/` . None existed in the cloned repository because `app/ml/models/` was mistakenly gitignored upstream.
- Missing Raw Training Data:** `backend/app/ml/data/raw/creditcard.csv` was missing from the repository, preventing local retraining.

- **TensorFlow & Python Dependency Incompatibilities:** The MLP model required TensorFlow and Keras, which were uninstalled. Subsequent installation pulled NumPy 2.x, which caused binary ABI incompatibilities with compiled C-extensions in SHAP (`_ARRAY_API` not found).

Remediation Steps Taken

1. **Dataset Acquisition:** Downloaded the full 143.84 MB Kaggle/HuggingFace credit card dataset (284,807 rows & 30 canonical features) to `backend/app/ml/data/raw/creditcard.csv`.
2. **Engineered Machine Learning Toolchain:** Installed TensorFlow 2.21 and Keras 3.15. Pinned `numpy==1.26.4` to ensure seamless C-extension compatibility across TensorFlow, Scikit-learn, XGBoost, LightGBM, and SHAP.
3. **Fixed Training Pipeline Bugs:**
 - `backend/app/ml/training/mlp.py`: Corrected a bug where `epochs=500` was hardcoded, ignoring the caller's parameter.
 - `backend/app/ml/training/extra_trees.py`: Enabled configurable `n_estimators=100` to prevent memory bloat and long training timeouts on resampled data.
4. **Full Model Training Execution:** Executed `train_models.py` with stratified 80/20 train/validation split, StandardScaler, and SMOTE resampling. Generated and verified all 6 artifacts:

Artifact	File	Size	Role
Feature Schema	<code>schema.json</code>	380 B	Canonical 30-feature schema enforcement
Feature Scaler	<code>scaler.pkl</code>	1.7 KB	Fitted StandardScaler on training fold
Base Model 1	<code>extra_trees.pkl</code>	56.8 MB	Randomized decision trees for non-linear interactions
Base Model 2	<code>mlp.keras</code>	238 KB	Swish-activated Deep MLP Neural Network
Meta Learner	<code>xgboost_meta.pkl</code>	15.4 KB	Gradient boosted stacking aggregator
Validation Metrics	<code>eval_metrics.json</code>	341 B	ROC-AUC: 0.9480 , Precision: 1.0000 , F1: 0.4444

5. **Fixed SHAP Explainability Engine:** In `backend/app/ml/explainability/engine.py`, patched `explain_local_shap()` to reliably extract 1D positive-class SHAP contribution arrays across both list and 3D ndarray formats returned by SHAP 0.45.0.

3. Frontend Navigation & RBAC Fixes

Users registering via the UI are assigned the `Customer` role by default. When navigating or loading the dashboard, customer users experienced 404 errors and 403 Forbidden console errors.

1. Added Dedicated Unauthorized Page

Fixed

Created `frontend/src/pages/UnauthorizedPage.tsx` with high-fidelity UI and registered route `/unauthorized` in `App.tsx`. Unauthorized access attempts now display a polished access-denied page rather than falling through to a 404 Not Found.

2. Role-Aware Sidebar Navigation

Fixed

Updated `frontend/src/components/layout/Sidebar.tsx` to filter navigation items based on the active role:

- **Customer:** Dashboard, My Transactions, Dispute Center.
- **Fraud Analyst:** Analyst Workspace, Live Transactions, Risk Calculator, Explainability Studio, Model Registry.
- **Administrator:** Complete administrative suite including Telemetry, Audit Logs, and System Settings.

3. Eliminated 403 Network Errors on Customer Dashboard

Fixed

In `frontend/src/pages/CustomerDashboard.tsx`, wrapped privileged telemetry and audit log queries with an `isPrivileged` check. Standard customer sessions no longer trigger failed calls to `/ops/audit` or `/telemetry/metrics`.

4. Topbar User Profile & Active Logout

Fixed

Updated `frontend/src/components/layout/Topbar.tsx` so the profile avatar displays the user's authentic initials (e.g. "AN", "AD") and added a functional sign-out action that clears JWT storage and navigates to `/login`.

4. Backend Configuration & Database Seeding

- **Virtual Environment:** Configured Python 3.11 virtual environment under `backend/venv`.
- **Secure Secrets:** Generated cryptographically strong 32-byte JWT and App secret keys in `backend/.env`.
- **SQLite Database:** Configured zero-dependency local database (`fraudshield_dev.db`) and ran database migrations (`flask db upgrade`).
- **Seeding Script Repair:** Fixed `backend/scripts/seed_dev_db.py` by aligning role enums (`RoLeEnum.ANALYST`) and stripping obsolete model parameters.
- **Pre-Warmed Inference:** The backend automatically pre-warms the ML engine in a background thread upon startup, eliminating first-request latency.

5. Test Suite Verification & Live Validation

Layer	Runner	Total Tests	Passed	Result
Backend (API, Auth, ML, Security)	pytest	94	94	100% Passed
Frontend (Components, Hooks, Pages)	vitest	16	16	100% Passed

Live Endpoint Verifications

```
POST /api/v1/auth/login → 200 OK (JWT Access Token Generated) POST /api/v1/predict/single → 200 OK (Ensemble Probability: 0.4513, Risk: Review Required) GET /api/v1/explainability/transaction/<id> → 200 OK (30 SHAP Feature Attributions) GET /api/v1/explainability/compare → 200 OK (Status: verified, ROC-AUC: 0.9480)
```

6. GitHub Repository Deployment

The local repository was reinitialized with a clean Git history authored by **KISHORE NANDHAN**, configured with proper ignore filters, and pushed to GitHub.

```
git branch -M main git remote add origin https://github.com/KISHORE-NANDHAN/Credit-Card-Fraud-Detection.git git push -u origin main
```

GitHub Repository: <https://github.com/KISHORE-NANDHAN/Credit-Card-Fraud-Detection.git>

All production ensemble artifacts (`schema.json` , `scaler.pkl` , `extra_trees.pkl` , `mlp.keras` , `xgboost_meta.pkl` , `eval_metrics.json`) are tracked in the repository so new clones work out of the box.

7. Access Credentials & Run Instructions

Available Demo Accounts

Role	Email	Password	Permissions
Administrator	admin@fraudshield.ai	Admin@SecurePass2026!	Full access to all dashboards, models, explainability, telemetry, and audit logs
Fraud Analyst	analyst@fraudshield.ai	Demo@12345	Analyst Workspace, Live Transactions, Risk Calculator, Explainability Studio, Model Registry
Customer	Register any account	Chosen password	Customer Dashboard, My Transactions, Dispute Center

Quick Start Commands

```
# 1. Start Backend (Terminal 1) cd backend .\venv\Scripts\python.exe run.py # Running on http://127.0.0.1:5000 # 2. Start Frontend (Terminal 2) cd frontend npm run dev # Running on http://localhost:5173
```